Dynamic Predicate Matchers vs. Comparing to true/false



One of the goals of TDD is to help you design your APIs. For this reason, when we're writing specs *for* a predicate method, we like to call the method directly and compare the return value explicitly with true or false. If we're testing a method named success?, we'll say expect(subject.success?).to be true in our examples.

On the other hand, when a predicate method isn't what we're directly testing but instead just happens to be available on a returned object, we like to use a dynamic predicate matcher. For instance, when you tested the Ledger#record method in the expense tracker project earlier in this book, we suggested that you write an expectation like expect(result).to be_success.

Satisfaction

Sometimes, you'll have a complicated condition that can't be expressed with any of the matchers we've seen so far. For these cases, RSpec provides the satisfy matcher. To use it, you wrap up your pass/fail logic into a block and hand that block to satisfy:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb

```
expect(1).to satisfy { |number| number.odd? }
```

RSpec passes the subject of the expectation, 1, into the block—which supports any arbitrary logic you can think up.

We like to think of satisfy as an *adapter*: it wraps any chunk of Ruby code and adapts it to RSpec's matcher protocol. This ability is useful when building a composed matcher expression:

11-matchers-included-in-rspec-expectations/01/primitive_matchers.rb

```
expect([1, 2, 3]).to include(an_object_satisfying(&:even?))
```

Here, we're using satisfy's alias, an_object_satisfying with a composed matcher expression. We're also saving a bit of verbiage by creating the block implicitly, using Ruby's Symbol#to_proc.[4]

As flexible as satisfy is, we still favor purpose-built matchers. The latter provide more specific, helpful failure messages.

4. http://ruby-doc.org/core-2.4.1/Symbol.html#method-i-to_proc